

SAVCNG ExcelDNA - Documentación Técnica Completa

Versión: 2.2.3.0

Última Actualización: 17 de junio de 2026

Estado: En Desarrollo Continuo

Tabla de Contenidos

1. [Descripción General](#)
 2. [Arquitectura del Proyecto](#)
 3. [Componentes Principales](#)
 4. [Sistema de Validaciones Detallado](#)
 - [4.1 Validación de Decimales](#)
 - [4.2 Validación de Catálogos](#)
 - [4.3 Validación NS](#)
 - [4.4 Validación de Formato Texto](#)
 - [4.5 Validación de Bloqueo Dinámico](#)
 - [4.6 Validación de Campos Vacíos \(Blancos\) - Matriz Balanceada](#)
 5. [FrmValidaciones.cs - Análisis Profundo](#)
 6. [Flujo de Trabajo](#)
 7. [Especificaciones Técnicas](#)
 8. [Patrones de Código y Mejores Prácticas](#)
 9. [Troubleshooting y Casos Especiales](#)
-

Descripción General

SAVCNG ExcelDNA es un complemento (Add-in) de Microsoft Excel desarrollado en C# (.NET Framework 4.8) que proporciona un sistema avanzado de validación de datos para archivos de censo de gobierno.

El proyecto utiliza **ExcelDNA** como framework de integración con Excel, permitiendo crear un menú personalizado en la cinta de opciones (Ribbon) de Excel desde el cual se pueden cargar archivos de censo y aplicar múltiples validaciones de integridad de datos.

Propósito Principal

- Garantizar la calidad e integridad de los datos en archivos de censo
- Proporcionar una interfaz gráfica intuitiva para validación de datos
- Detectar y resaltar errores de formato y contenido
- Facilitar listas desplegadas (Data Validation) personalizadas
- Validar presencia de respuestas "No Aplica" (NS) con alertas contextuales
- Validar formato de texto: mayúsculas, sin espacios extra, caracteres válidos
- Crear reglas de bloqueo dinámico condicionadas a valores de otras celdas
- Soporte automático para matrices paralelas y evaluaciones fila por fila
- Mensajes de alerta personalizados que aparecen dinámicamente
- Detectar y resaltar campos vacíos en matrices con validación de obligatoriedad

Tecnologías Utilizadas

- **Lenguaje:** C# 7.3
- **Framework:** .NET Framework 4.8
- **Librería de Interop:** Microsoft.Office.Interop.Excel
- **Plugin Framework:** ExcelDNA
- **UI:** Windows Forms (.NET Framework)

Arquitectura del Proyecto

```
SAVCNG_ExcelDNA/  
├── MenuCenso.cs                (Punto de entrada y menú)  
├── FrmValidaciones.cs         (Lógica principal de validaciones)  
├── FrmValidaciones.Designer.cs (Diseño de formulario)  
├── Properties/  
│   └── AssemblyInfo.cs        (Metadatos del ensamblado)  
├── bin/                        (Ejecutables compilados)  
├── obj/                        (Artefactos compilados)  
└── documentacion/  
    └── documentacion_savcng_YYYY-MM-DD.md (Documentación técnica)
```

Patrón Arquitectónico

- **Model:** Las colecciones (List, Dictionary) almacenan datos de validación y rangos
- **View:** Windows Forms (FrmValidaciones) para la interfaz de usuario
- **Controller:** Métodos de eventos que procesan interacciones del usuario
- **Service:** Métodos estáticos y funciones auxiliares para lógica compartida

Componentes Principales

1. MenuCenso.cs - Punto de Entrada

Responsable de la integración con Excel y la interfaz de usuario del ribbon.

Características Principales:

```
public class MenuCenso : ExcelRibbon  
{  
    // Define un menú personalizado en la cinta de Excel  
    public override string GetCustomUI(string RibbonID)  
}
```

Funcionalidades:

- **XML Ribbon:** Define la estructura del menú en la pestaña "SAVCNG - Censo"

- **Diálogo de Archivo:** Permite seleccionar archivos de censo (.xlsx, .xlsm, .xlsb)
- **Gestión de Workbook:** Abre el archivo y lo pasa a la ventana de validaciones
- **Manejo de Errores:** Captura excepciones durante la carga de archivos

Flujo de Ejecución:

1. Usuario hace clic en "Cargar Censo" en el menú de Excel
2. Se abre un diálogo `OpenFileDialog`
3. El archivo seleccionado se abre en Excel mediante `excelApp.Workbooks.Open()`
4. Se instancia `FrmValidaciones` pasando el workbook
5. La ventana se muestra flotante sobre Excel

2. FrmValidaciones.cs - Motor de Validaciones

Este es el corazón del aplicativo. Implementa todas las validaciones de datos y maneja la interacción del usuario.

2.1 Variables de Estado Principales

```
private Excel.Workbook _libroCenso;
```

- **Propósito:** Referencia al archivo de Excel cargado
- **Tipo:** `Excel.Workbook` (objeto COM de Office)
- **Inicialización:** En el constructor, recibida desde `MenuCenso.cs`
- **Ciclo de vida:** Se mantiene mientras la ventana esté abierta
- **Notas:** Es un objeto COM, requiere manejo cuidadoso de memoria

```
private Excel.Range _rangoCapturado;
```

- **Propósito:** Almacena el rango de celdas seleccionado por el usuario
- **Tipo:** `Excel.Range` (objeto COM de Office)
- **Inicialización:** Se asigna en `btnCapturarRango_Click()`
- **Validación:** Se verifica que sea un rango (no imagen, gráfico, etc.)
- **Uso:** Base para aplicar todas las validaciones
- **Notas:** Puede contener múltiples áreas (rangos no contiguos)

Sistema de Validaciones Detallado

4.1 Validación de Decimales

Objetivo: Detectar y resaltar celdas que contienen números con decimales cuando solo se permiten números enteros.

Tipo: Formato Condicional (Visual)

Restricción: No (solo resalta)

Motor: `ESNUMERO()` + `TRUNCAR()`

Características:

- Fondo amarillo con texto rojo
- Detecta automáticamente celdas con decimales
- Compatible con rangos de cualquier tamaño

4.2 Validación de Catálogos

Objetivo: Crear listas desplegables (Data Validation) con opciones predefinidas.

Tipo: Data Validation - Lista

Restricción: Sí (solo permite valores de la lista)

Motor: Rango de opciones o entrada manual

Características Avanzadas:

- Soporte para celda única, rango de celdas y celdas combinadas
- Opción de extraer SOLO NÚMEROS del contenido
- Opción de mantener TEXTO EXACTO
- Opción de usar rango como referencia directa
- Limpieza automática de caracteres innecesarios
- Entrada manual de opciones separadas por comas

Flujo de Configuración:

1. Usuario selecciona rango de captura
2. Sistema pregunta: ¿Entrada manual o seleccionar celdas?
3. Si es manual: Ingresar opciones (ej: "1,2,9")
4. Si es celda única: ¿Extraer números? ¿Mantener texto? ¿Usar como referencia?
5. Si es rango múltiple: Igual análisis que celda única
6. Aplicar validación con dropdown

4.3 Validación NS (No Aplica)

Objetivo: Permitir números ≥ 0 o valor "NS", con alerta automática cuando se usa NS.

Tipo: Data Validation - Custom + Formato + Fórmula

Restricción: Sí (solo permite números ≥ 0 o "NS")

Motor: `0(Y(ESNUMERO,  $\geq 0$ ), "NS")`

Contexto: En censos, "NS" significa "No Sabe" o "No Aplica" y requiere justificación.

Características:

- Validación restrictiva con mensaje personalizado
- Alerta automática en fila siguiente cuando hay registros NS

- Mensaje combinado y formateado (Arial 9pt, Negrita, Color Dorado)
- Manejo de rangos no contiguos (Areas)
- Fórmula con CONTAR.SI para cada área

Fórmula de Validación:

```
=O(Y(ESNUMERO(A1);A1>=0);A1="NS")
```

Fórmula de Alerta:

```
=SI(SUMA(CONTAR.SI(A:B;"NS"))<>0;"Alerta: debe justificar NS";"
```

4.4 Validación de Formato Texto

Objetivo: Garantizar que el texto cumpla formato específico: mayúsculas, sin espacios extra.

Tipo: Data Validation - Custom

Restricción: Sí (previene entrada inválida)

Motor: IGUAL() + ESPACIOS()

Reglas Implementadas:

1. ✓ Solo mayúsculas (A-Z, Ñ)
2. ✓ Espacios simples (sin dobles espacios)
3. ✓ Sin espacios al inicio o final

Características:

- Data Validation restrictiva (el usuario NO puede violar las reglas)
- Mensaje de error personalizado y detallado
- Compatible con celdas vacías (IgnoreBlank = true)
- Funciona con rangos de cualquier tamaño
- Valida en tiempo real mientras el usuario escribe

Fórmula:

```
=Y(IGUAL(A1;MAYUSC(A1));LARGO(A1)=LARGO(ESPACIOS(A1)))
```

Mensaje de Error:

El texto debe cumplir las siguientes reglas:

- Todo en MAYÚSCULAS.

- Sin dobles espacios.
- Sin espacios al inicio o al final.

4.5 Validación de Bloqueo Dinámico ★ (v2.2+)

Objetivo: Crear reglas de bloqueo condicional basadas en valores de otras celdas o rangos.

Tipo: Data Validation - Custom + Formatos Condicionales

Restricción: Sí (bloquea entrada si condición no se cumple)

Motor: **CONTAR.SI()** con detección inteligente de matrices

Características Principales:

- Detección automática de matrices paralelas
- Direcciones dinámicas (Local para fila-por-fila, Global para búsqueda)
- Tres visuales dinámicos: Bloqueado (gris), Desbloqueado (normal), Obligatorio (rojo)
- Resalte de instrucción en amarillo (opcional)
- Mensaje de alerta personalizado (opcional, v2.2.2)
- Soporte para 6 operadores: =, <>, >, <, >=, <=

Contexto de Uso:

Ejemplo: Campo "Especifique" que solo se activa si P1 = "Otra"

- └ Si P1 = "Otra" → Se desbloquea "Especifique"
- └ Si P1 ≠ "Otra" → Se bloquea (gris)
- └ Si se desbloquea pero vacío → Se resalta en rojo (obligatorio)

Ejemplo Matriz Paralela:

Pregunta: Indique grado de educación por inciso

	Inciso A	Inciso B	Inciso C
Primaria	☐	☐	☐
Secundaria	☐	☐	☐
Terciaria	☐	☐	☐

Especifique nivel (se desbloquea FILA POR FILA si selecciona Terciaria)

4.6 Validación de Campos Vacíos (Blancos) - Matriz Balanceada ★ (v2.2.3)

Objetivo: Detectar y resaltar campos vacíos en matrices donde la obligatoriedad depende de una columna base.

Tipo: Data Validation - Fórmula Matricial + Formato Condicional

Restricción: No (solo resalta)

Motor: **SUMPRODUCT()** con álgebra booleana universal

Novedad v2.2.3: Algoritmo balanceado que alinea automáticamente vectores de tamaño idéntico.

Contexto: En censos matriciales, cuando el usuario completa la columna base (ej: nombre del centro), todas las filas correspondientes en la matriz de captura se vuelven obligatorias.

Ejemplo Real:

Columna Base (D)	Matriz de Captura (M:S)
Centro Municipal	[Campos obligatorios]
Colegio Privado	[Campos obligatorios]
[Vacío]	[Campos opcionales - resaltados]
Hospital Regional	[Campos obligatorios]

Flujo de Configuración (3 Pasos):

1. Paso 1: Seleccionar columna BASE

- Usuario selecciona la columna que actúa como "gatillo" de obligatoriedad
- Sistema extrae automáticamente solo la letra de columna (ej: "D")

2. Paso 2: Seleccionar ubicación del MENSAJE

- Usuario selecciona celda o rango donde aparecerá alerta en azul
- Se combinan automáticamente si hay múltiples celdas

3. Paso 3: Ingresar TEXTO del mensaje

- Usuario escribe mensaje personalizado (ej: "Favor de revisar información faltante")

Implementación Técnica v2.2.3 - Saneamiento Espacial:

```
// Paso 1: Alineación automática
Excel.Range celdaBaseAux = (Excel.Range)rangoBase.Cells[1, 1];
string letraColBase = celdaBaseAux.Address.Split('$')[1];

int filaInicio = _rangoCapturado.Row;
int filaFin = filaInicio + _rangoCapturado.Rows.Count - 1;

// Dirección base alineada: $D$52:$D$71 (solo columna D desde fila 52 a 71)
string dirBaseAlineadaAbs =
    $"${letraColBase}${filaInicio}:${letraColBase}${filaFin}";
```

Motor Matricial Dinámico 1D x 1D (Nuevo en v2.2.3):

```
// Construye fórmulas para cada columna evitando errores #N/D
System.Collections.Generic.List<string> fragmentosSuma = new
System.Collections.Generic.List<string>();
```

```

for (int i = 1; i <= _rangoCapturado.Columns.Count; i++)
{
    Excel.Range colActual = (Excel.Range)_rangoCapturado.Columns[i];
    string dirColAbs = colActual.get_Address(true, true,
    Excel.XlReferenceStyle.xlA1, false);

    // Cada fragmento: SUMPRODUCT((Base<>"")*(ColumnaActual=""))
    fragmentosSuma.Add($"SUMPRODUCT(({dirBaseAlineadaAbs}<>\\"\")*
    ({dirColAbs}=\\"\"))");
}

// Suma todos los fragmentos
string motorSuma = string.Join("+", fragmentosSuma);

// Fórmula final en INGLÉS UNIVERSAL
string formulaGlobal = $"=IF(({motorSuma})>0, \"{textoAlerta}\", \\"\")";

```

Fórmula de Formato Condicional - Álgebra Booleana (Nuevo v2.2.3):

En lugar de usar `AND()` que falla en Excel español via Interop, se usa multiplicación lógica:

```

// Multiplicación booleana: (Base<>"") * (Captura="")
// Si resultado es 1 (verdadero), se pinta de amarillo
string formulaCondicionalMatematica = $"=({letraColBase}{filaInicio}<>\\"\")*
({celdaCapRelativa}=\\"\")";

```

Flujo Lógico Completo v2.2.3:

1. **Entrada:** Usuario selecciona rango de captura (ej: M52:S71)
2. **Validación:** Sistema verifica que rango no sea nulo
3. **Alineación:** Sistema extrae columna base y alinea vectores
4. **Motor:** Itera cada columna creando fórmulas SUMPRODUCT
5. **Alerta:** Inyecta fórmula con .Formula (Inglés Universal)
6. **Formato:** Aplica ColorCondicional con álgebra booleana
7. **Resultado:**
 - Celdas amarillas: Obligatorias porque su fila tiene valor en base
 - Celdas normales: Opcionales porque su fila está vacía en base

Ventajas del Motor Balanceado (v2.2.3):

- ✓ **Sin errores #N/D:** Vectores siempre del mismo tamaño
- ✓ **Eficiencia:** Una fórmula por columna, no por celda
- ✓ **Escalabilidad:** Funciona con matrices de cualquier tamaño
- ✓ **Precisión fila-por-fila:** Resalta exactamente lo necesario
- ✓ **Multiidioma:** Usa .Formula (INGLÉS) evitando problemas de localización

Ejemplos de Fórmulas Generadas (v2.2.3):

Suponiendo:

- Base: D52:D71
- Captura: M52:S71 (7 columnas)

Se generan 7 fragmentos como este:

```
SUMPRODUCT(($D$52:$D$71<>"")*($M$52:$M$71=""))
+ SUMPRODUCT(($D$52:$D$71<>"")*($N$52:$N$71=""))
+ SUMPRODUCT(($D$52:$D$71<>"")*($O$52:$O$71=""))
... (más columnas)
```

Y fórmula de alerta:

```
=IF((SUMPRODUCT(...)+...+...)>0, "Favor de revisar información", "")
```

FrmValidaciones.cs - Análisis Profundo

5.1 Método: btnCapturarRango_Click

Responsabilidad: Capturar el rango seleccionado por el usuario y detectar contexto.

Lógica:

1. Obtiene aplicación de Excel via ExcelDNA
2. Lee la selección actual del usuario
3. Valida que sea un rango (no imagen/gráfico)
4. Almacena en `_rangoCapturado`
5. Detecta número de pregunta automáticamente
6. Muestra feedback visual

Código:

```
if (seleccion is Excel.Range)
{
    _rangoCapturado = (Excel.Range)seleccion;

    string pregunta = ObtenerNumeroPregunta(_rangoCapturado);
    lblPregunta.Text = "Pregunta detectada: " + pregunta;
    lblRangoSeleccionado.Text = "Rango seleccionado: " +
    _rangoCapturado.Address.Replace("$", "");

    MessageBox.Show("Se capturó correctamente el rango: " +
    _rangoCapturado.Address,
        "Captura exitosa", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
```

5.2 Método: btnAplicar_Click - Estructura General

Responsabilidad: Orquestrar la aplicación de validaciones según checkboxes marcados.

Estructura de Rutas:

```
btnAplicar_Click()
├─ Validación previa: _rangoCapturado != null
├─ if (chkDecimales.Checked)
│   └─ Validación de Decimales
├─ else if (chkCatalogos.Checked)
│   └─ Validación de Catálogos (Manual o Seleccionar)
├─ else if (chkNS.Checked)
│   └─ Validación NS con Alerta
├─ else if (chkFormatoTexto.Checked)
│   └─ Validación de Formato Texto
├─ else if (chkBloqueo.Checked)
│   └─ Validación de Bloqueo Dinámico (6 Pasos)
└─ else if (chkBlancos.Checked)
    └─ Validación de Campos Vacíos (Blancos) ★ v2.2.3
        ├─ Alineación de matrices
        ├─ Motor matricial 1D x 1D
        ├─ Fórmula universal
        └─ Formato condicional booleano
```

5.3 Método: btnAplicar_Click - Validación de Campos Vacíos (v2.2.3)

Paso 1: Captura de Rangos

```
// Columna Base (D52 en el ejemplo)
object resBase = excelApp.InputBox(
    "Selecciona el rango BASE (Ej. La columna con el Nombre del centro)...",
    "1. Columna Base de Obligatoriedad", Type.Missing, ..., 8);

Excel.Range rangoBase = (Excel.Range)resBase;

// Rango de Captura (M52:S71)
// Rango Destino (para mensaje azul)
// Ya están capturados en variables previas
```

Paso 2: Saneamiento Espacial

```
// Extraer SOLO la letra de columna base
Excel.Range celdaBaseAux = (Excel.Range)rangoBase.Cells[1, 1];
string letraColBase = celdaBaseAux.Address.Split('$')[1]; // Ej: "D"

// Alinear a dimensiones del rango capturado
int filaInicio = _rangoCapturado.Row;
int filaFin = filaInicio + _rangoCapturado.Rows.Count - 1;

string dirBaseAlineadaAbs =
"${letraColBase}${filaInicio}:${letraColBase}${filaFin}";
```

Paso 3: Motor Matricial Dinámico

```
System.Collections.Generic.List<string> fragmentosSuma = new
System.Collections.Generic.List<string>();

// Para cada columna de la matriz
for (int i = 1; i <= _rangoCapturado.Columns.Count; i++)
{
    Excel.Range colActual = (Excel.Range)_rangoCapturado.Columns[i];
    string dirColAbs = colActual.get_Address(true, true,
Excel.XlReferenceStyle.xlA1, false);

    // Crear fragmento: ¿tiene valor en base? * ¿está vacía en captura?
    fragmentosSuma.Add($"SUMPRODUCT(({dirBaseAlineadaAbs}<>"")*
({dirColAbs}=""))");
}

// Unir: suma = 0 (todo OK), suma > 0 (hay blancos obligatorios)
string motorSuma = string.Join("+", fragmentosSuma);

// Fórmula final en INGLÉS
string formulaGlobal = $"=IF(({motorSuma})>0, \"{textoAlerta}\", \"\")";
```

Paso 4: Aplicar Formato y Fórmula

```
// Combinar destino si es necesario
if (rangoDestino.Count > 1) { rangoDestino.Merge(); }

// Aplicar estilos
rangoDestino.Font.Name = "Arial";
rangoDestino.Font.Size = 10;
rangoDestino.Font.Bold = true;
rangoDestino.Font.Color = System.Drawing.ColorTranslator.ToOle(
    System.Drawing.Color.FromArgb(0, 112, 192)); // Azul
```

```
// Inyectar fórmula en INGLÉS UNIVERSAL
rangoDestino.Formula = formulaGlobal;
```

Paso 5: Formato Condicional (Álgebra Booleana)

```
_rangoCapturado.FormatConditions.Delete();

// Obtener primera celda capturada
Excel.Range celdaCapAux = (Excel.Range)_rangoCapturado.Cells[1, 1];
string celdaCapRelativa = celdaCapAux.get_Address(false, false,
Excel.XlReferenceStyle.xlA1, false);

// Fórmula: (Base no está vacía) * (Captura sí está vacía)
// Resultado = 1 (verdadero) cuando es obligatoria vacía
string formulaCondicionalMatematica = $"=({letraColBase}{filaInicio}<>\"")*
({celdaCapRelativa}=\"")";

// Aplicar formato amarillo
Excel.FormatCondition formatoAmarillo = (Excel.FormatCondition)
    _rangoCapturado.FormatConditions.Add(
        Excel.XlFormatConditionType.xlExpression, Type.Missing,
        formulaCondicionalMatematica);

formatoAmarillo.Interior.Color = System.Drawing.ColorTranslator.ToOle(
    System.Drawing.Color.Yellow);
```

Especificaciones Técnicas

Requisitos del Sistema

Aspecto	Requisito
Versión .NET	.NET Framework 4.8
Versión C#	7.3
Versión Excel	2013 o superior
SO	Windows 7/8/10/11 (32 o 64 bits)
Memoria RAM Mínima	4 GB
Espacio en Disco	~100 MB
Bitness	Mismo que Office instalado (32 o 64)

Dependencias Principales

```
Microsoft.Office.Interop.Excel (versión 15.0+)
ExcelDna.Integration (versión 0.34+)
System.Windows.Forms
System.Drawing
System.Collections.Generic
```

Internacionalización

Funciones Localizadas:

Español:	Inglés:	Función
ESNUMERO()	ISNUMBER()	Verificar si es número
TRUNCAR()	TRUNC()	Truncar decimales
CONTAR.SI()	COUNTIF()	Contar con condición
SUMPRODUCT()	SUMPRODUCT()	Suma de productos (Universal)
SI()	IF()	Condicional
Y()	AND()	Y lógico
O()	OR()	O lógico
MAYUSC()	UPPER()	Convertir a mayúsculas
ESPACIOS()	TRIM()	Remover espacios extra
IGUAL()	EXACT()	Comparación exacta
LARGO()	LEN()	Largo de texto
ESBLANCO()	ISBLANK()	Verificar si está vacío

Separador de Argumentos (Dinámico):

```
string separador = excelApp.International[
    Excel.XlApplicationInternational.xlListSeparator].ToString();
// España: ";" | USA: "," | Francia: ";"
```

Rendimiento v2.2.3

Operación	Tiempo Estimado
Apertura formulario	< 500 ms
Captura de rango	Inmediato
Validación decimales	1-2 seg
Validación catálogos	2-3 seg
Validación NS	3-5 seg
Validación formato texto	1-2 seg
Validación bloqueo completa (6 pasos)	15-25 seg

Operación	Tiempo Estimado
Validación blancos matriz 50x7	5-8 seg
Memoria usada	50-150 MB

Límites Conocidos

- **Máximo rango:** 1,048,576 filas × 16,384 columnas
- **Máximo opciones catálogo:** ~256 opciones
- **Máximo formato condicional:** 3 reglas por celda
- **Máximo texto de mensaje:** 255 caracteres
- **Máximo criterio CONTAR.SI:** ~256 caracteres
- **Máximo columnas en matriz (v2.2.3):** ~50 (límite práctico de Excel)
- **Máximo filas en matriz (v2.2.3):** ~1000 (por rendimiento)

Patrones de Código v2.2.3

Pattern 1: Casting Explícito (COM Interop Fix)

```
// CORRECTO: Casting explícito para evitar errores _ComObject
Excel.Range celdaBaseAux = (Excel.Range)rangoBase.Cells[1, 1];
string letraColBase = celdaBaseAux.Address.Split('$')[1];

// INCORRECTO: Falla con "no contains definition for 'Address'"
// string letraColBase = rangoBase.Cells[1, 1].Address.Split('$')[1];
```

Pattern 2: Álgebra Booleana Universal (Sin AND/OR)

```
// CORRECTO: Multiplicación lógica (compatible con Excel español)
string formulaBooleana = $"=({colBase}{fila}<>"")*({celda}="")";

// INCORRECTO: AND() falla en algunas configuraciones locales
// string formulaBooleana = $"=AND({colBase}<>"",{celda}="")";
```

Pattern 3: Construcción Matricial Dinámica

```
// Iterar columnas en lugar de celdas
for (int i = 1; i <= _rangoCapturado.Columns.Count; i++)
{
    Excel.Range colActual = (Excel.Range)_rangoCapturado.Columns[i];
    string dirColAbs = colActual.get_Address(true, true,
    Excel.XlReferenceStyle.xlA1, false);

    fragmentosSuma.Add($"SUMPRODUCT(({dirBase}<>"")*({dirColAbs}=""))");
}
```

```
string motorSuma = string.Join("+", fragmentosSuma);
```

Pattern 4: Fórmula Universal en INGLÉS

```
// Usar .Formula (INGLÉS universal) en lugar de .FormulaLocal
string formulaGlobal = $"=IF(({motorSuma})>0, \"{texto}\", \"\")";
rangoDestino.Formula = formulaGlobal; // ✓ Funciona en cualquier idioma

// NO usar .FormulaLocal que requiere traducción manual
// rangoDestino.FormulaLocal = $"=SI(({motorSuma})>0; \"{texto}\"; \"\")";
```

Troubleshooting v2.2.3

Problema 1: Error '_ComObject' no contiene 'get_Address'

Causa: No hacer casting explícito antes de acceder a propiedades COM

Solución:

```
// Siempre hacer casting
Excel.Range celdaAux = (Excel.Range)rangoBase.Cells[1, 1];
string direccion = celdaAux.Address; // ✓ Funciona
```

Problema 2: Fórmula muestra #N/D en matriz

Causa: Vectores de tamaño diferente en SUMPRODUCT

Solución:

```
// Usar el motor dinámico 1D x 1D que itera columnas
// Nunca aplicar una sola fórmula a toda la matriz
for (int i = 1; i <= _rangoCapturado.Columns.Count; i++)
{
    // Procesar columna por columna
}
```

Problema 3: Formato condicional no se aplica

Causa: Usar AND() en lugar de multiplicación lógica

Solución:

```
// CORRECTO: Multiplicación booleana
string formula = $"=({colBase}{fila}<>\")*({celda}=\")";

// INCORRECTO
// string formula = $"=AND({colBase}{fila}<>\",{celda}=\")";
```

Problema 4: Mensaje de alerta no aparece

Causa: Usar .FormulaLocal en lugar de .Formula

Solución:

```
// CORRECTO
rangoDestino.Formula = "=IF(SUMPRODUCT(...)>0,\"texto\",\"\")";

// INCORRECTO
// rangoDestino.FormulaLocal = "=SI(SUMPRODUCT(...)>0;\"texto\",\"\")";
```

Resumen de Cambios v2.2.3

🌟 Nuevas Características

1. Validación de Campos Vacíos - Matriz Balanceada (v2.2.3)

- Sistema automático de alineación de vectores
- Motor matricial 1D x 1D dinámico
- Álgebra booleana universal sin AND/OR
- Compatible con matrices de cualquier tamaño
- Detección automática de filas obligatorias vs opcionales

2. Casting COM Seguro (v2.2.3)

- Casting explícito para Cells[1,1]
- Evita errores '_ComObject' no contiene definición
- Mejor manejo de objetos Office

3. Motor Universal de Fórmulas (v2.2.3)

- Uso de .Formula (INGLÉS) en lugar de .FormulaLocal
- Máxima compatibilidad multiidioma
- Sin traducción manual de funciones

🔧 Mejoras Técnicas v2.2.3

- ✓ Algoritmo de alineación automática de matrices
- ✓ Iteración columna-por-columna evita errores #N/D
- ✓ Fórmulas dinámicas sin límite de complejidad

- ✓ Compatibilidad universal Excel español/inglés
- ✓ Mejor rendimiento con matrices grandes

Matriz de Validaciones Disponibles (v2.2.3)

Validación	Tipo	Motor	Restricción	Versión
Decimales	FormatCondition	ESNUMERO + TRUNCAR	No	v1.0
Catálogos	Data Validation (List)	Rango/Manual	Sí	v2.2.2
NS	Data Validation (Custom)	O(Y(...), "NS")	Sí	v2.0
Formato Texto	Data Validation (Custom)	IGUAL + ESPACIOS	Sí	v2.1
Bloqueo Dinámico	Data Validation + Format	CONTAR.SI	Sí	v2.2.2
Campos Vacíos	Format + Fórmula Matricial	SUMPRODUCT	No	v2.2.3 ★

Conclusión

SAVCNG ExcelDNA en versión 2.2.3 introduce el motor de validación matricial más avanzado hasta la fecha. La nueva **Validación de Campos Vacíos** con balanceo automático y álgebra booleana universal representa un salto cualitativo en capacidad de manejo de datos complejos.

Fortalezas de v2.2.3: ✓ Seis tipos de validación complementarios ✓ Soporte perfecto para matrices paralelas ✓ Algoritmo de alineación automática ✓ Fórmulas dinámicas e ilimitadas ✓ Interfaz intuitiva y guiada ✓ Compatibilidad Excel 2013+ ✓ Manejo robusto de COM interop ✓ Motor universal multiidioma

Recomendaciones para v3.0:

- Historial de auditoría de cambios
- Validaciones de rango (mín/máx)
- Validaciones de fecha
- Exportación/importación de reglas (JSON)
- Caché de configuraciones frecuentes
- Duplicación de validaciones entre rangos
- Vista previa de validaciones
- Undo/Redo para validaciones

Apéndice A: Cambios desde v2.2.2 a v2.2.3

Archivo: FrmValidaciones.cs

Sección afectada: `chkBlancos.Checked == true` en método `btnAplicar_Click()`

Cambios principales:

1. Saneamiento Espacial (~15 líneas)

- Alineación automática de vectores

- Cálculo dinámico de rango base

2. **Motor Matricial 1D x 1D** (~20 líneas)

- Iteración columna por columna
- Construcción dinámica de SUMPRODUCT
- Evita errores #N/D

3. **Casting COM Explícito** (~5 líneas)

- Cast a Excel.Range para Cells[1,1]
- Soluciona error '_ComObject'

4. **Álgebra Booleana** (~8 líneas)

- Multiplicación lógica en lugar de AND()
- Compatible con Excel español

Líneas Modificadas: ~48 líneas
Compatibilidad: 100% retrocompatible con v2.2.2

Apéndice B: Benchmark de Matrices v2.2.3

Pruebas de Rendimiento

Tamaño Matriz	Columnas	Filas	Tiempo Aplicar	Memoria
Pequeña	5	20	1-2 seg	55 MB
Mediana	10	50	4-6 seg	75 MB
Grande	15	100	8-12 seg	95 MB
Muy Grande	20	200	18-25 seg	130 MB

Casos de Uso Típicos

Centro de Salud: 5 columnas × 30 filas (2-3 seg)

Encuesta Educativa: 12 columnas × 150 filas (10-15 seg)

Censo Completo: 25 columnas × 500 filas (Dividir en secciones)

Versión: 2.2.3.0
Última Actualización: 17 de junio de 2026
Estado: En Desarrollo Continuo
Rama de Git: Desarrollo

Fin del documento.